

Internet Protocols

Computer Systems Security

2025/26

LEI - UM

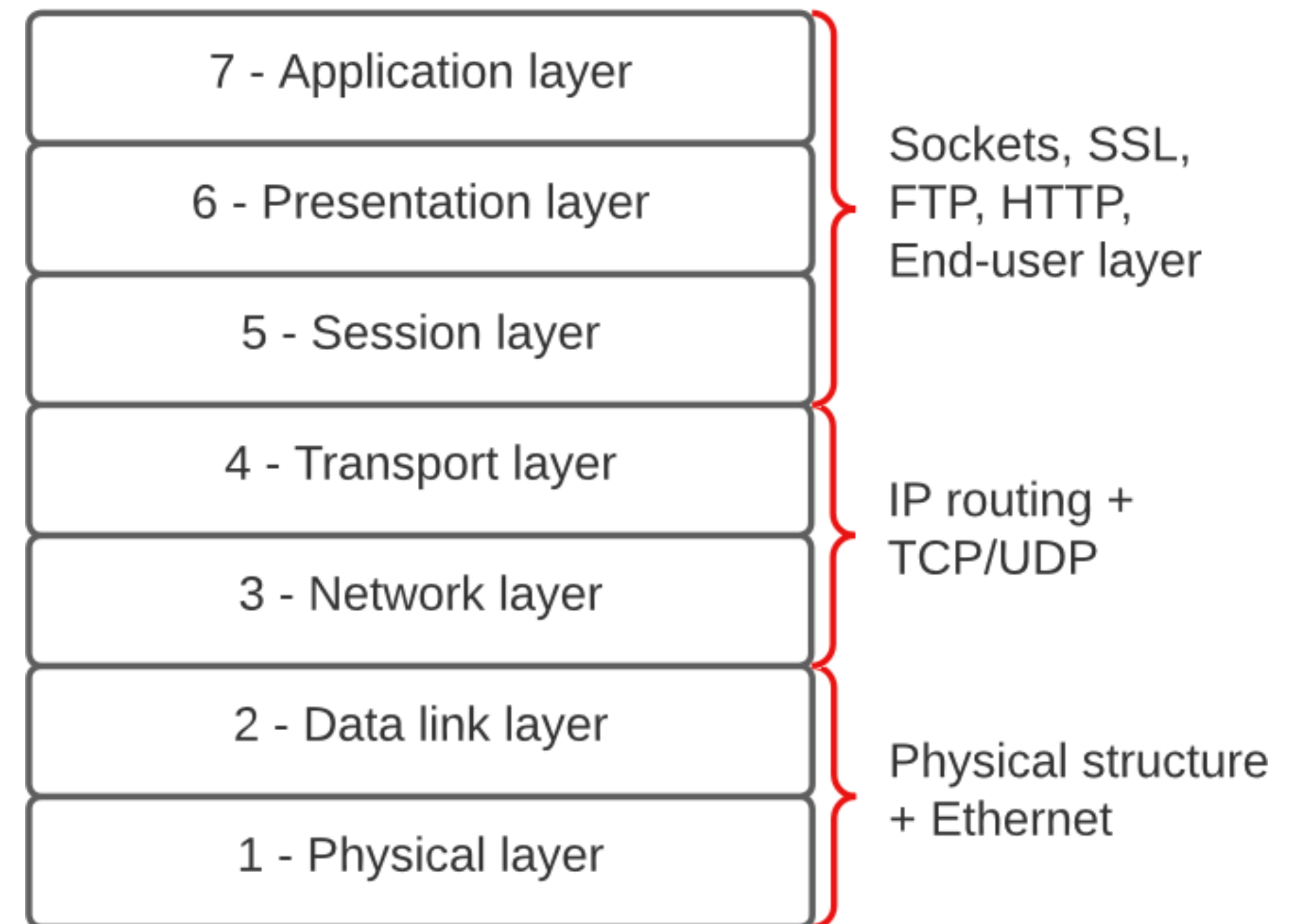
Hugo Pacheco hpacheco@di.uminho.pt

Security in the Internet

- In the Internet, cryptography is used everywhere for secure communications
- The World Wide Web is fundamentally a (highly distributed) client / server application running over the Internet
- We will see some protocols for:
 - **Communication:** establishing a secure channel
 - **Authentication:** Whether a user should be allowed access to a system
 - **Authorization:** Which actions a user can perform in the system

Communication Protocols

- Various protocols allow the establishment of a **secure channel**, ensuring:
 - Integrity of data
 - Entity Authentication (Server side or mutual authentication)
 - Confidentiality of data (optional)
- These protocols act on different OSI (Open Systems Interconnection) layers:
 - network layer (IPSec, VPNs);
 - transport layer (SSL, TLS);
 - application layer (SSH; S/MIME; PGP)



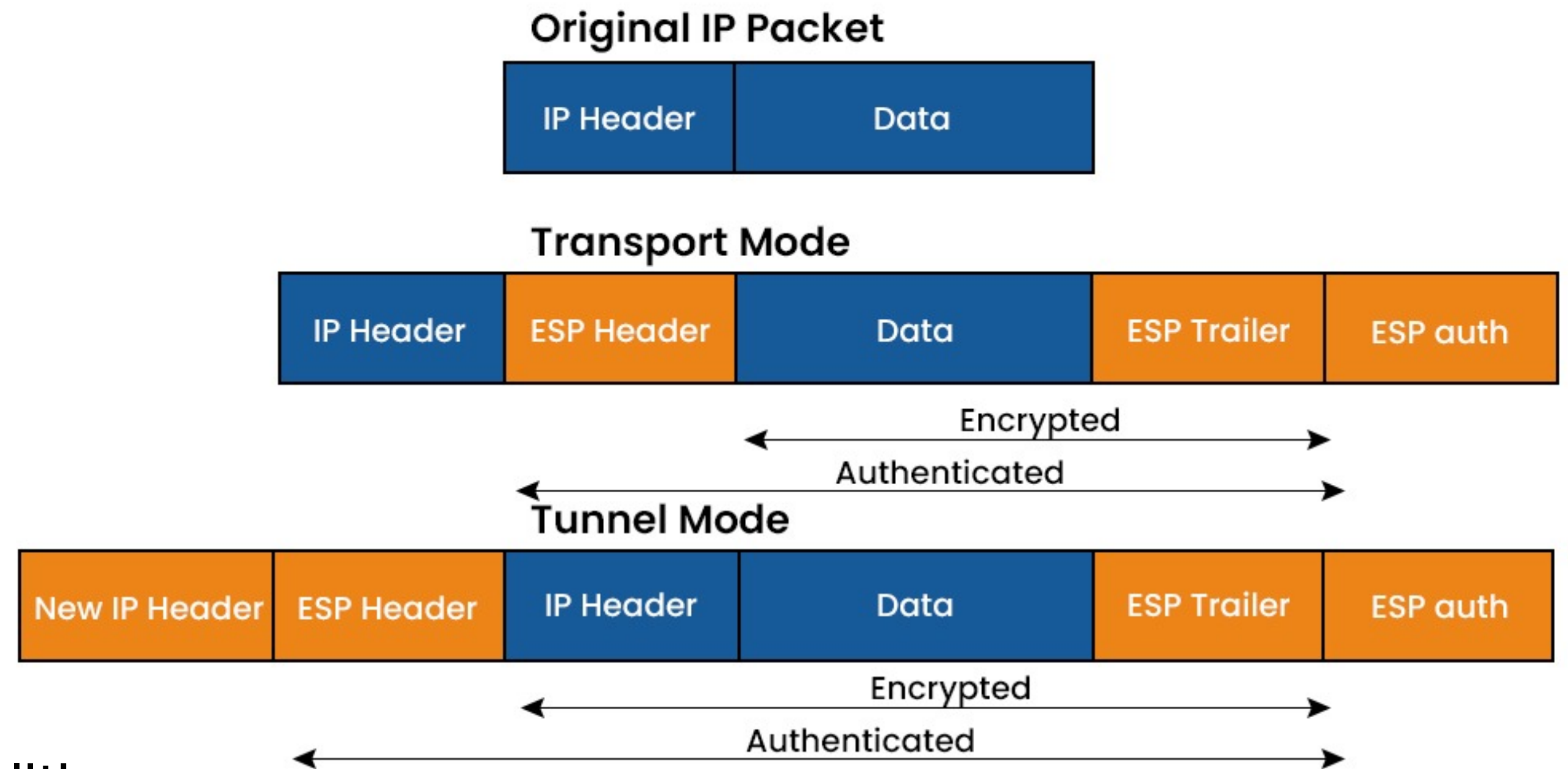
Communication Protocols

- Secure communication protocols typically include two phases
 - **Handshake (or key management):** negotiate parameters, asymmetric cryptographic algorithms and keys to setup a secure communication link
 - Before any data is transmitted
 - Usually the most complex step of the protocol
 - Occasionally, this phase may be replayed (e.g. rekeying)
 - **Secure communication link:** realises the secure channel using symmetric cryptography
- Other phases may be present for notifications, keep alive, etc

Communication

IPSec

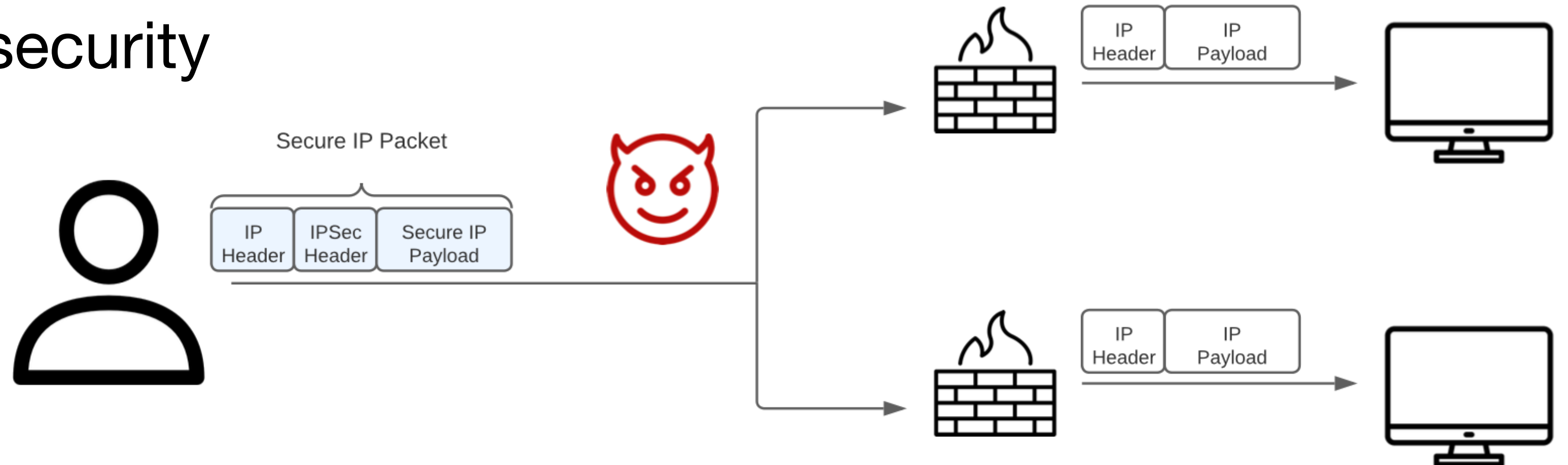
- IPSec refines the IP protocol
Included in IPv6, optional for IPv4
- **At the network layer (OS)**
Transparent to applications
- RFC specification quite complex
Extends IP with two main security functionalities:
 - **Authentication Header (AH):** integrity services at packet level, authentication of data origin and, optionally, protection against packet repetition
 - **Encapsulating Security Payload (ESP):** confidentiality (encryption) of all or only part of the traffic
- Offered at the network layer, but provides protection also to all higher layers
 - Web traffic iteratively encapsulates data: IP encapsulates TCP that encapsulates HTTP



IPSec

Applications

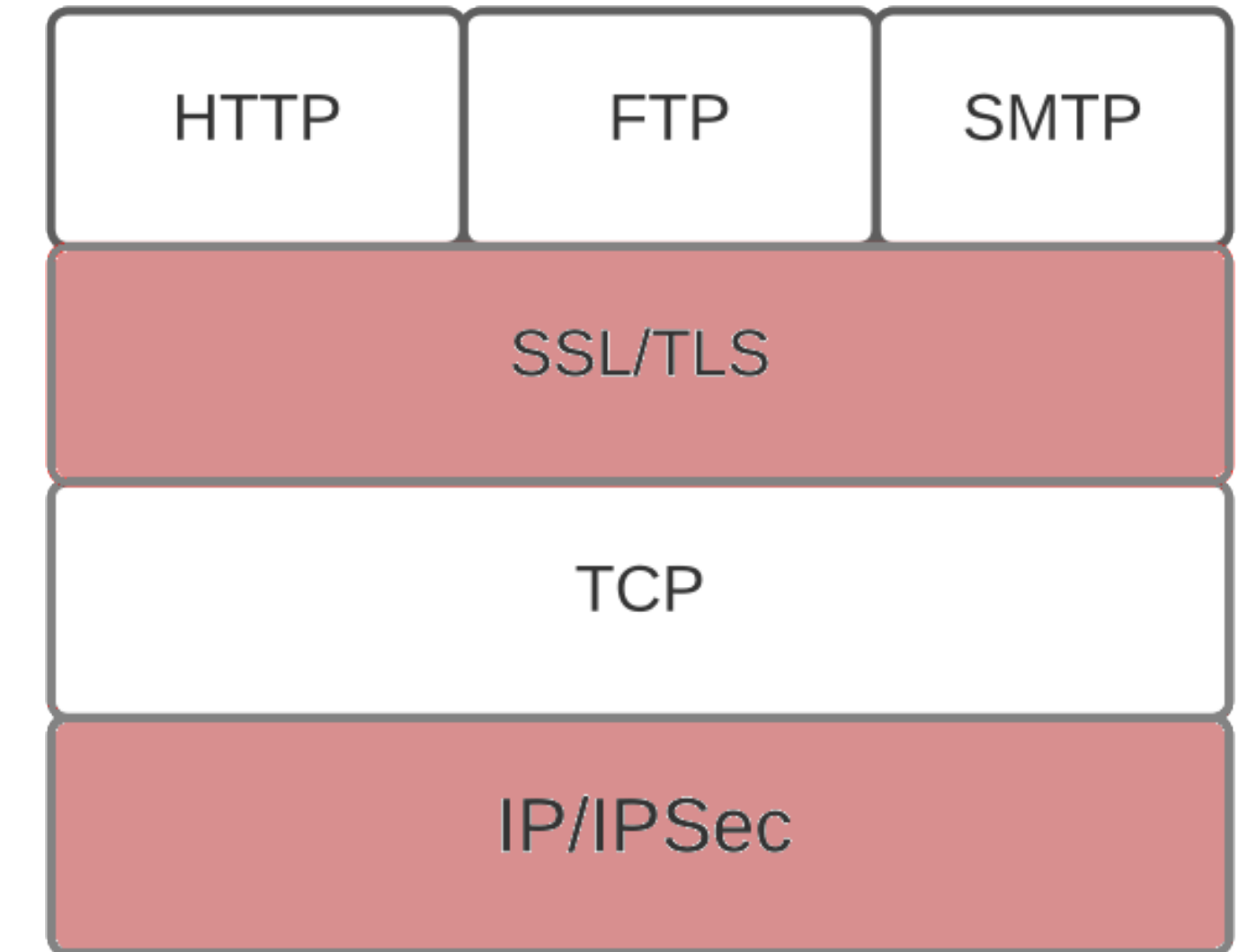
- IPSec thrives in applications where the same security is always necessary, and the same security techniques can be applied for all applications
 - From IP onward, everything is the same
 - All communications must be confidential and authenticated
- A typical use case is VPN security



Communication

SSL/TLS

- **SSL/TLS is a middleware between application and TCP**
Used for the majority of secure internet transactions today
- Transport Layer Security (TLS) evolved from the commercial Secure Sockets Layer (SSL), developed by Netscape
 - Improved configurability, protocols, security, etc
 - SSL v3 was adopted by the IETF under the name Transport Layer Security (TLS)
 - People often still call SSL to everything
- Implemented as part of the underlying protocol suite and embedded in specific packages
- Handshake supports **mutual authentication**, where the server and client request and validate each other's certificates



TLS

Architecture

- **TLS session**

- Persistent association between a client and a server
- Created by the handshake protocol
- Defines a set of crypto security parameters, shared among multiple connections
- Avoids expensive negotiation stages at the start of each connection

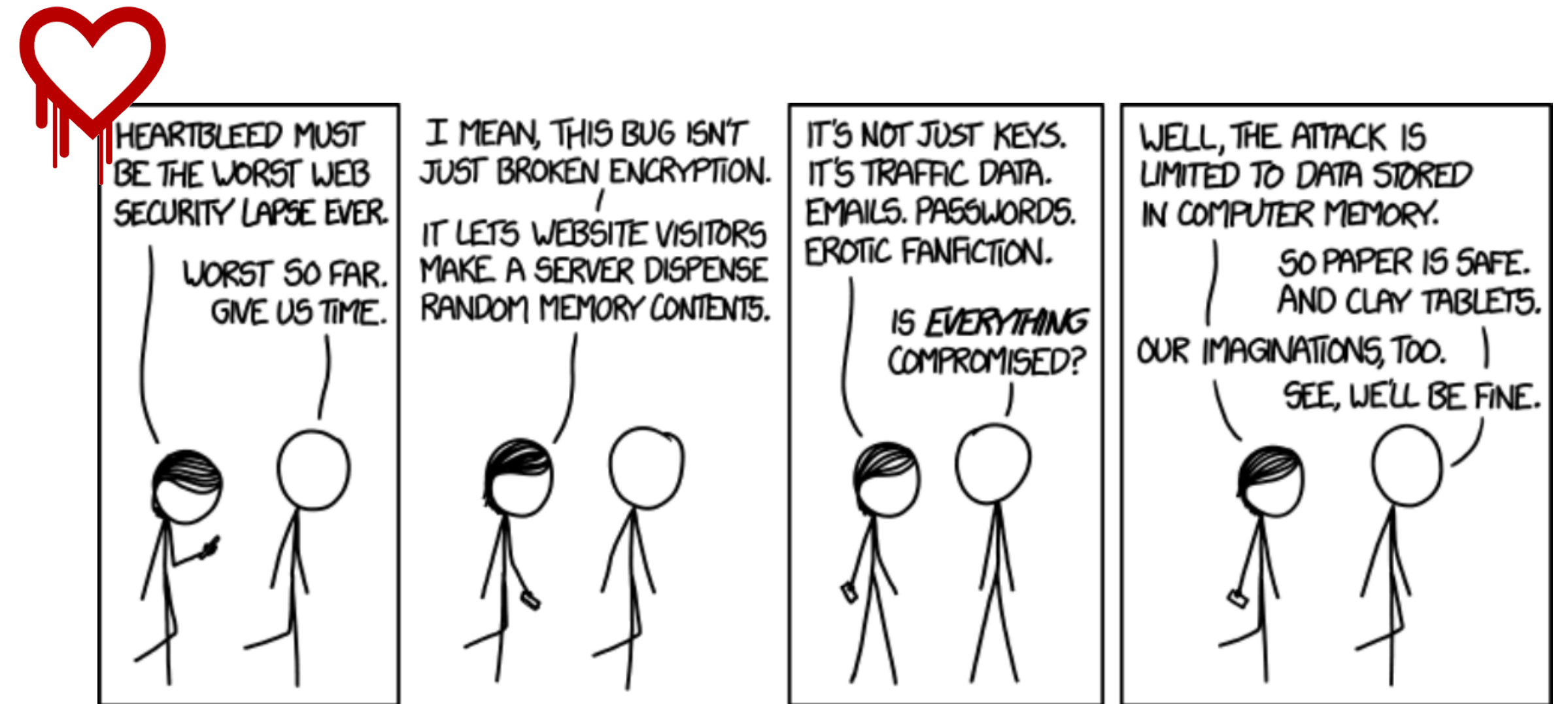
- **TLS connection**

- A transport that provides a suitable type of service
- For TLS, such connections are peer-to-peer
- Connections are transient
- Every connection is associated to one session, but connections may reuse the same session

TLS

Security

- History of attacks to SSL/TLS over the years is embarrassing
 - Attacks aimed at the protocol itself have been overcome by successive revisions
 - Appealing target: OpenSSL widely used (estimated 2/3 of web servers)
- The most recent attacks on SSL/TLS are mostly aimed at:
 - Less careful implementation or obscure features (e.g. **Heartbleed attack**)
 - Interaction with specific cipher suites (e.g. padding oracle attack in CBC mode)
- The sheer number of configuration options requires great care (see <https://ssllabs.com>)



TLS

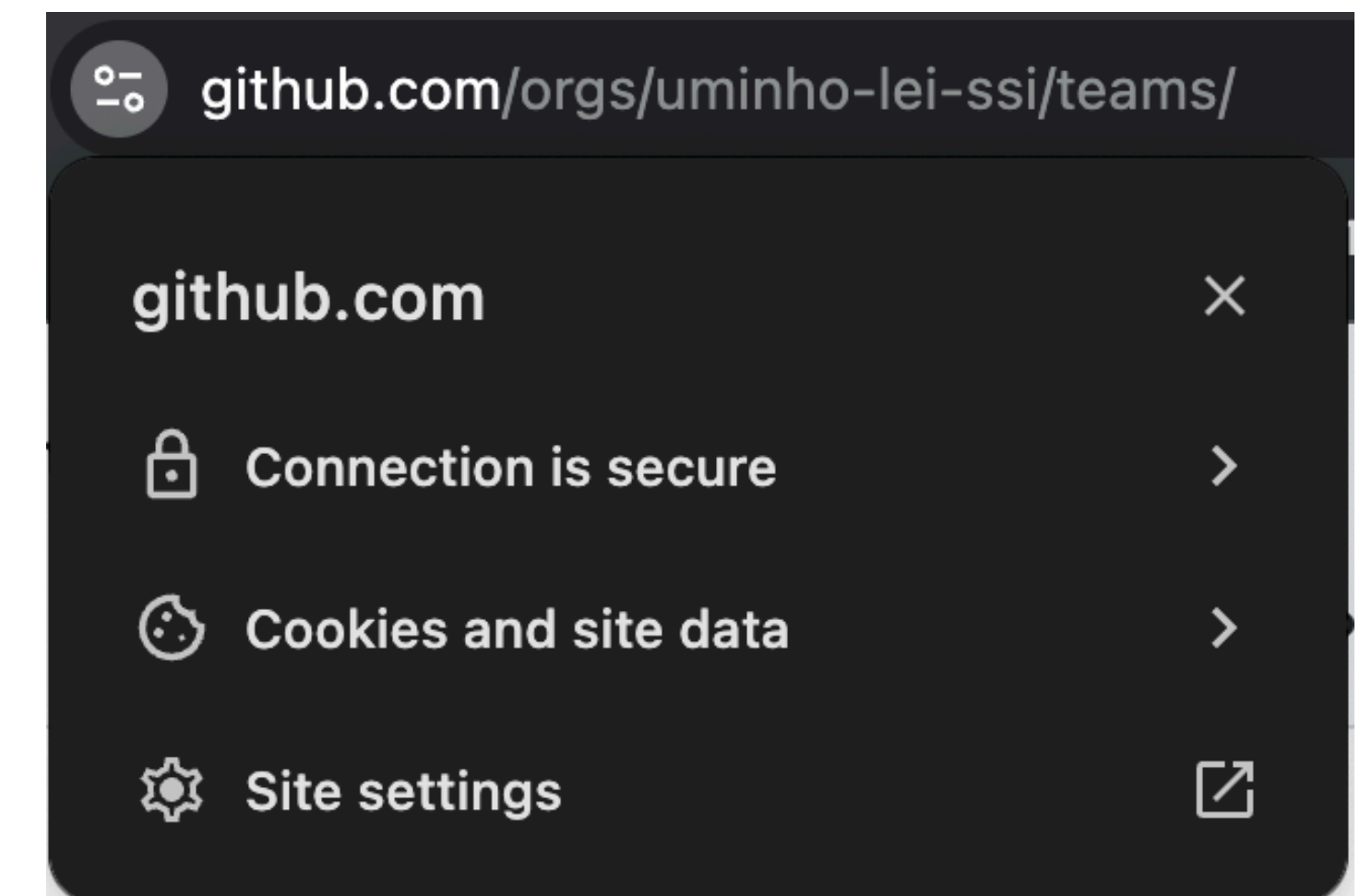
Version 1.3

- The current version has recently completed its final standardisation (RFC-8446)
 - Substantial redefinition of the protocol
 - Drops backward compatibility with previous versions
 - Greater efficiency and security
- Some features:
 - Simplified handshake (1 roundtrip)
 - Removes support for obscure or little-used features and cryptographic techniques (e.g. 3DES, RC4, MD5, SHA-1, AES-CBC, ...)
 - Supports only a restricted set of possibilities (DHE groups, AEAD modes; ...)

HTTPS

HTTP over SSL

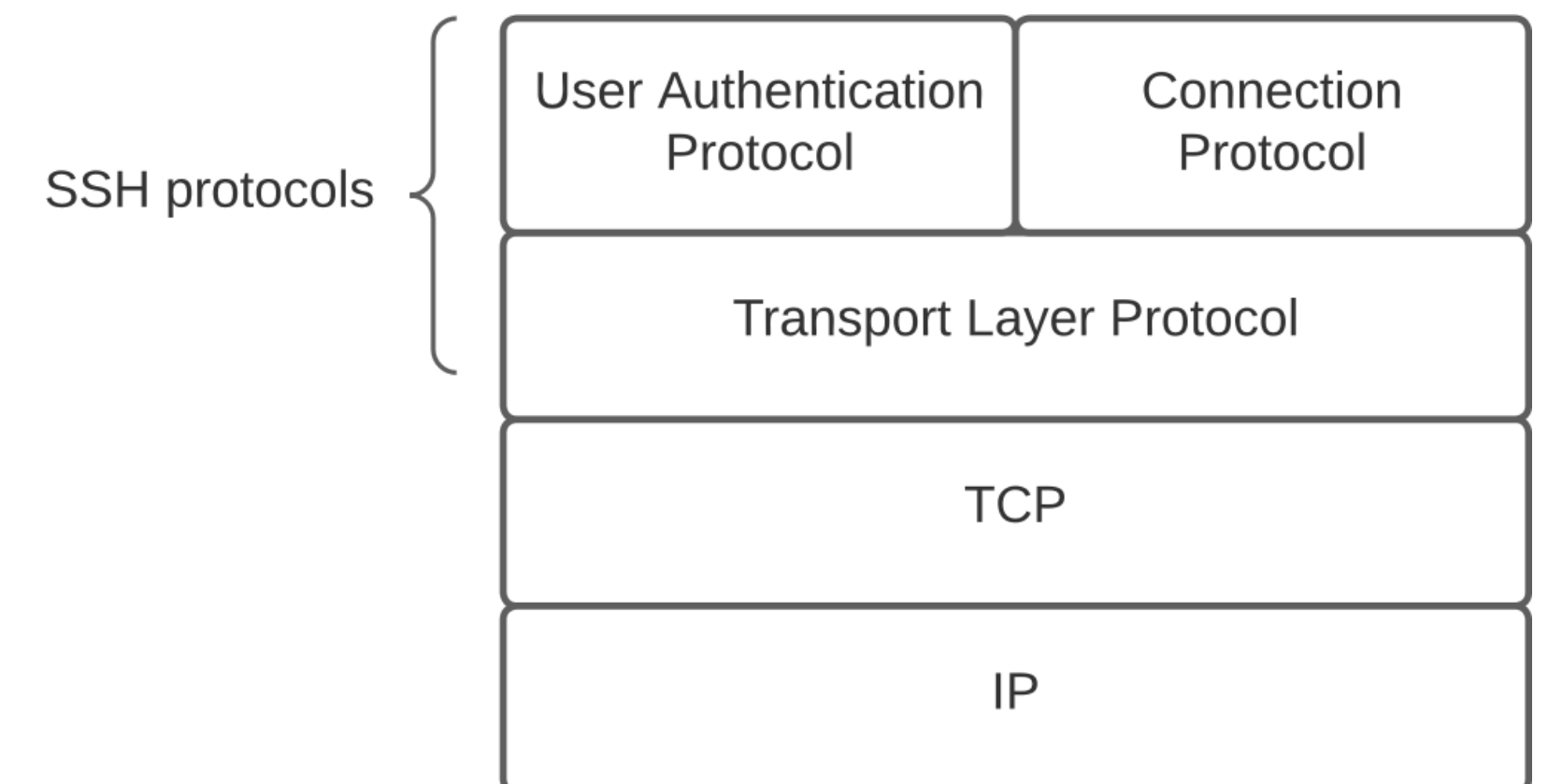
- HTTP+SSL to implement secure communication between a Web browser and a Web server
 - SSL in **server authentication** mode, where only the server certifies its identity
 - Agent acting as HTTP client also acts as the TLS client
 - Build into all modern Web browsers (URL addresses begin with https://)
- When HTTPS is used, the following elements are protected:
 - URL of requested document
 - Document contents
 - Contents of browser forms
 - Cookies sent from browser to server and vice-versa
 - Contents of HTTP header



Communication

SSH (Secure Shell Protocol)

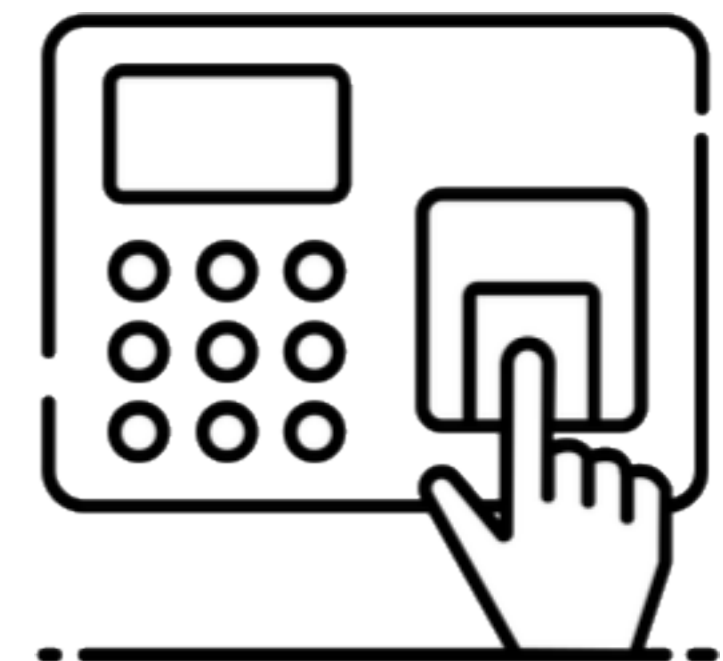
- Originally developed for Unix, now available on most OSes
- The *de facto* method to access remote resources
 - Provides an authenticated, encrypted path to the OS command line over the network
 - Replacement for insecure utilities such as Telnet, rlogin, rsh
 - Protects against spoofing attacks and modification of data
- Independent protocol stack, different trust assumptions
 - SSL/TLS: PKI certificates
 - SSH: TOFU (Trust-On-First-Use)



Authentication

Multi-Factor Authentication (MFA)

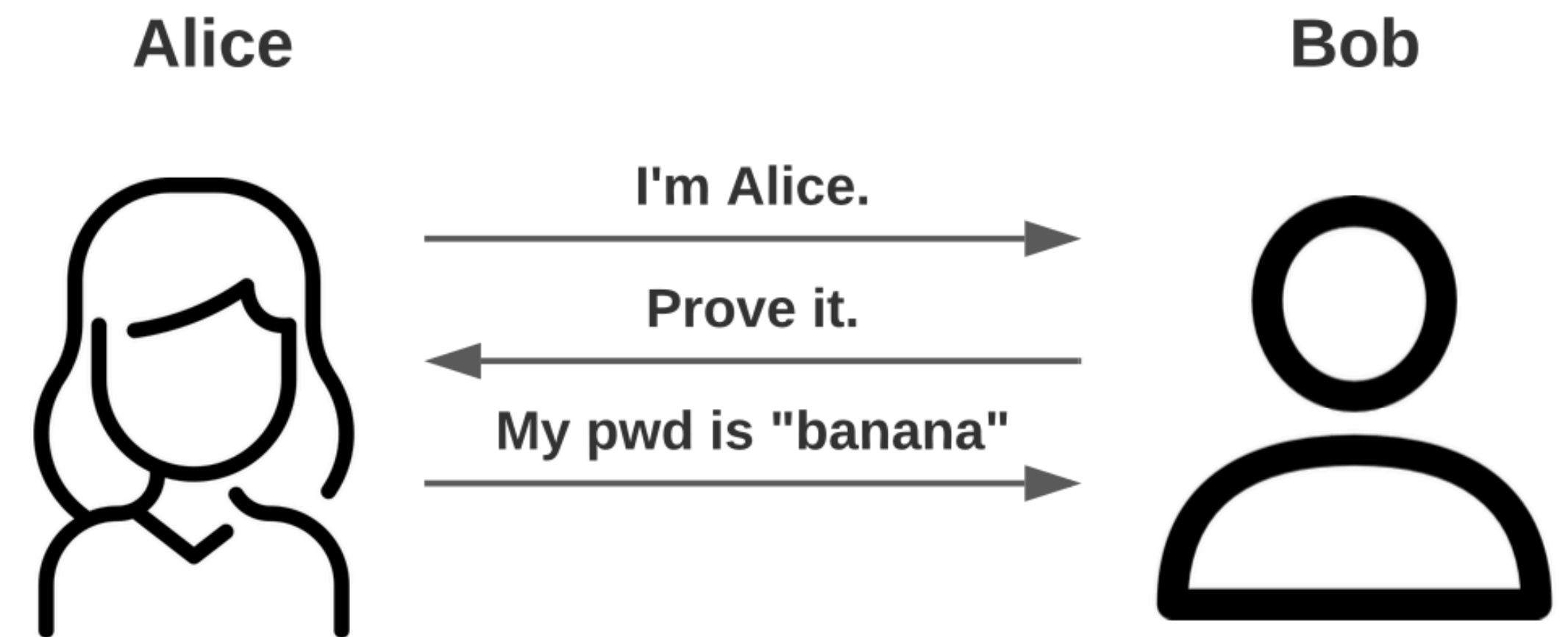
- Something you **know**
 - Passwords (single or multi-word)
 - Security question
- Something you **have**
 - Smart Cards
 - Crypto Tokens
- Something you **are**
 - Fingerprint
 - Other biological data



Authentication

A naive protocol

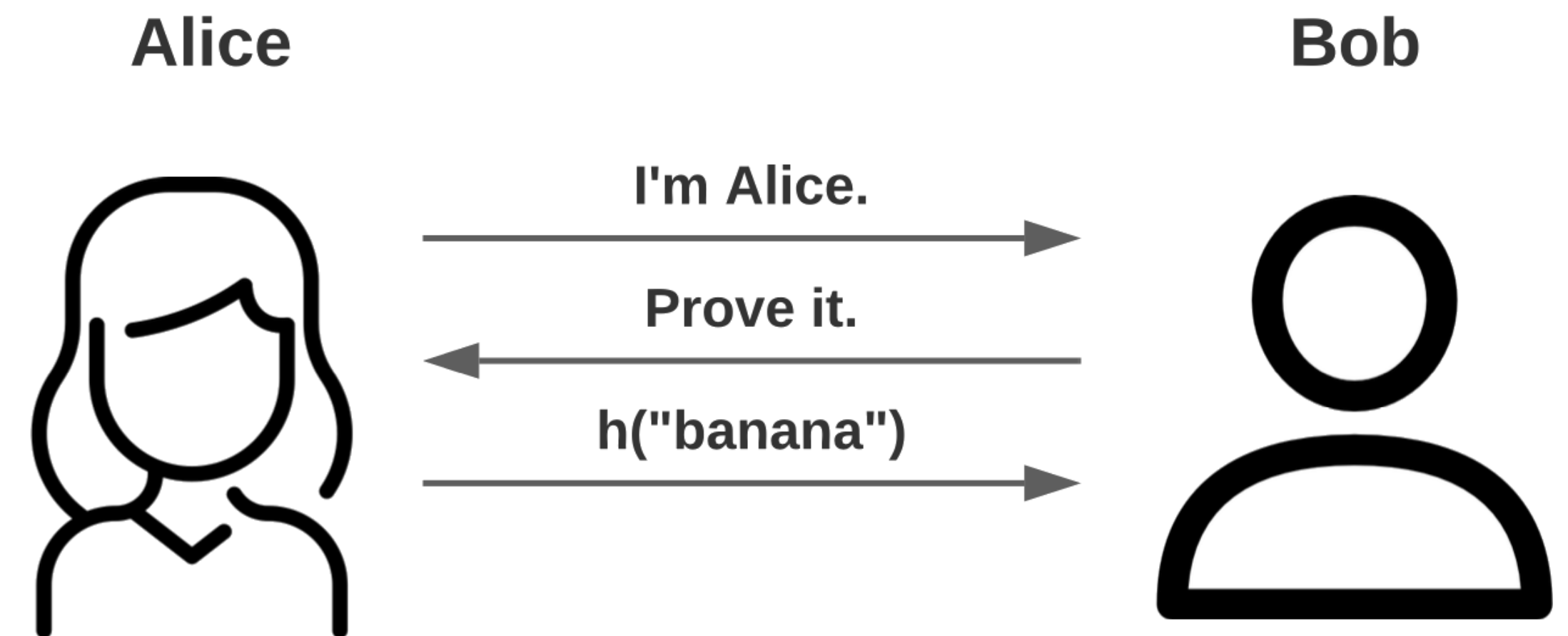
- Simple and perhaps acceptable for a local machine
- Highly insecure for a networked system
 - Stolen password if not encrypted
 - Subject to replay attacks
 - Attacker can repeat messages or communication patterns
 - Bob must (explicitly) know Alice's password



Authentication

A still naive protocol

- + Hides Alice's password
 - From Bob and from a network attacker
- Still subject to replay attacks
 - A network attacker can prove to be Alice



Authentication

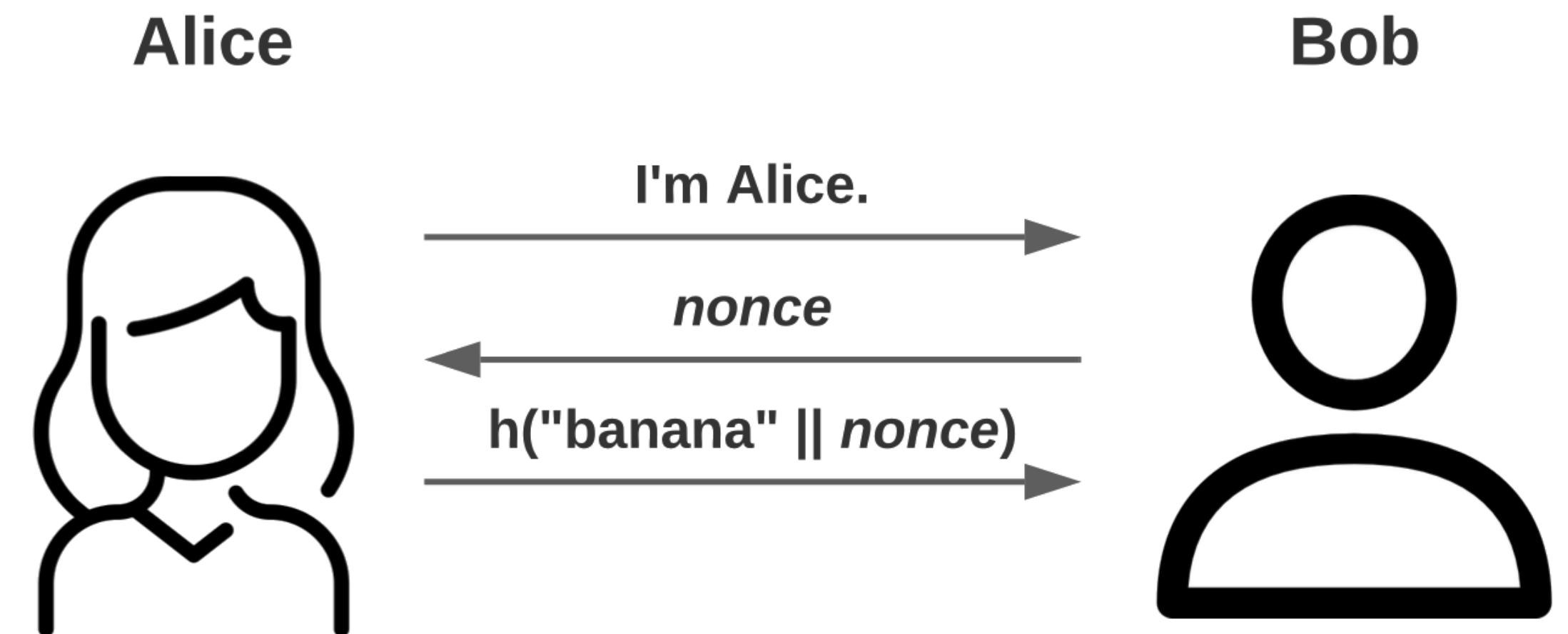
Challenge-response

- To prevent replay attacks, we leverage a technique called **challenge-response**
- Scheme (for Alice to authenticate with Bob):
 - Bob sends a challenge to Alice
 - Alice responds to the challenge according to her password
- A good challenge is such that:
 - Replay is not possible
 - How exactly? **Using a Nonce**
 - Only Alice can provide the correct response
 - Bob can (efficiently) verify the response

Challenge-Response

Symmetric protocol

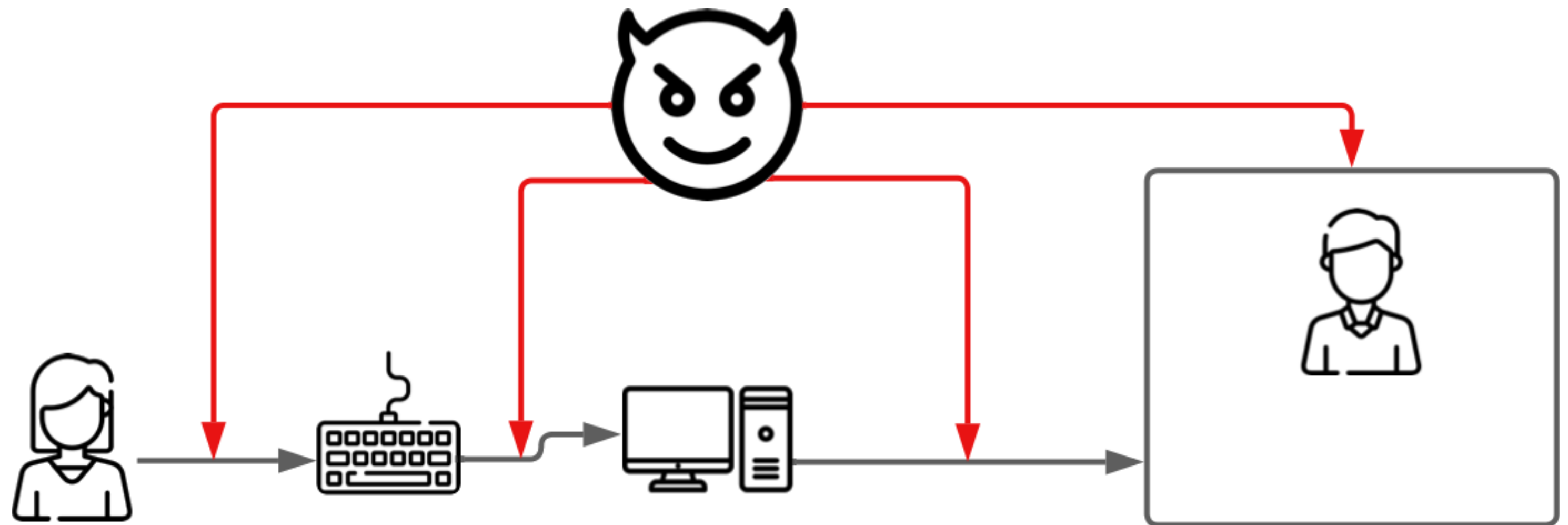
- The challenge is a Nonce
- The response is a hash of the message plus the Nonce
- + Prevents replay attacks
 - Hashes won't repeat (assuming fresh Nonce and collision-resistant hash function)
- Bob must know Alice's password to verify (hence symmetric)



Authentication

Passwords

- **The weakest link is still the password!**
 - Typically low entropy
- Attacks come in many flavors
 - In-person
 - Keyloggers
 - Network packet sniffing
 - Server (Bob) hacking
 - Data leaks, dictionary attacks, etc



Challenge-Response

Possibilities

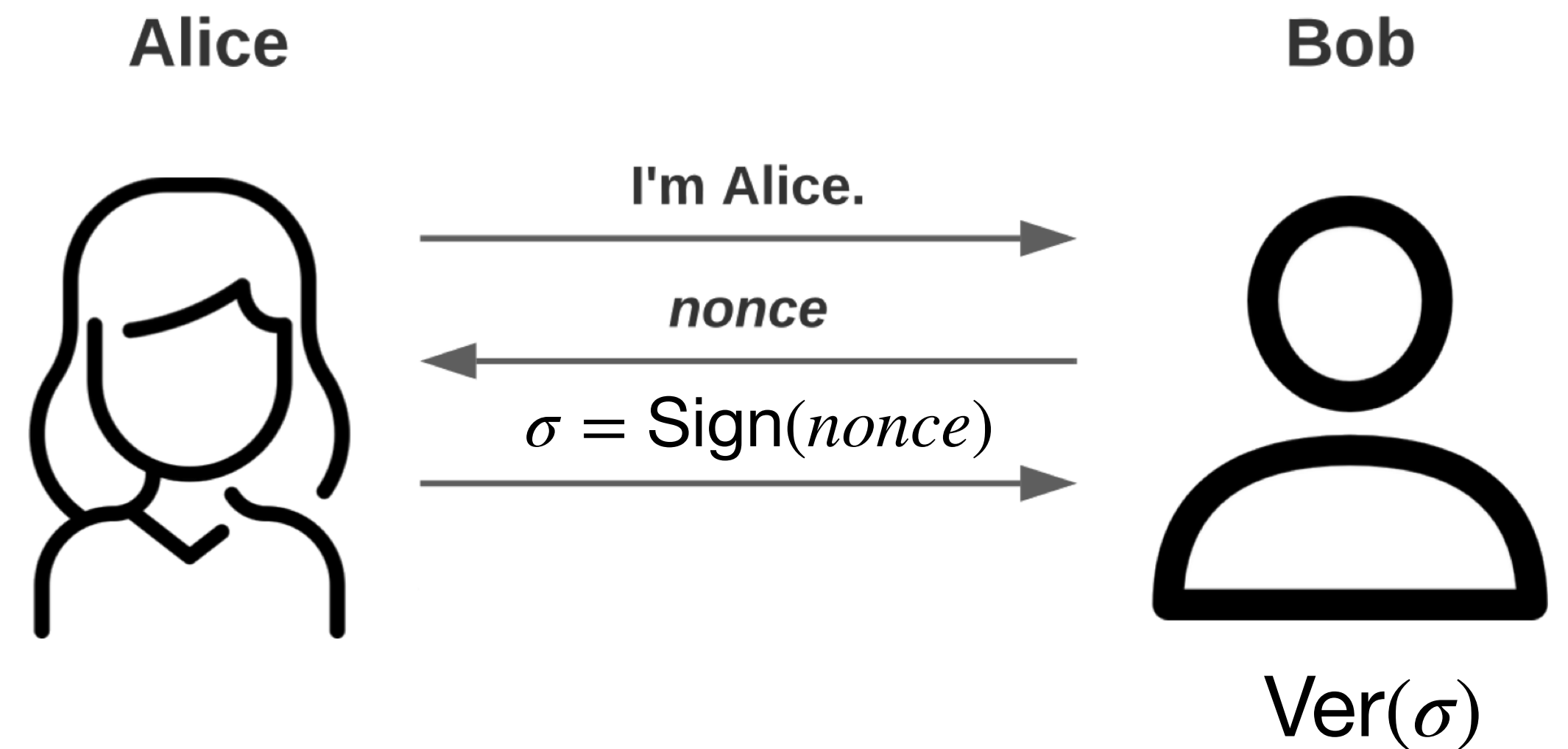
- A “great” challenge is such that:
 - **Bob must not know a hard secret**
 - Replay is not possible
 - **Only Alice can provide the correct response**
 - **Bob can (efficiently) verify the response**
- **Smells like digital signatures!**



Challenge-Response

Asymmetric protocol

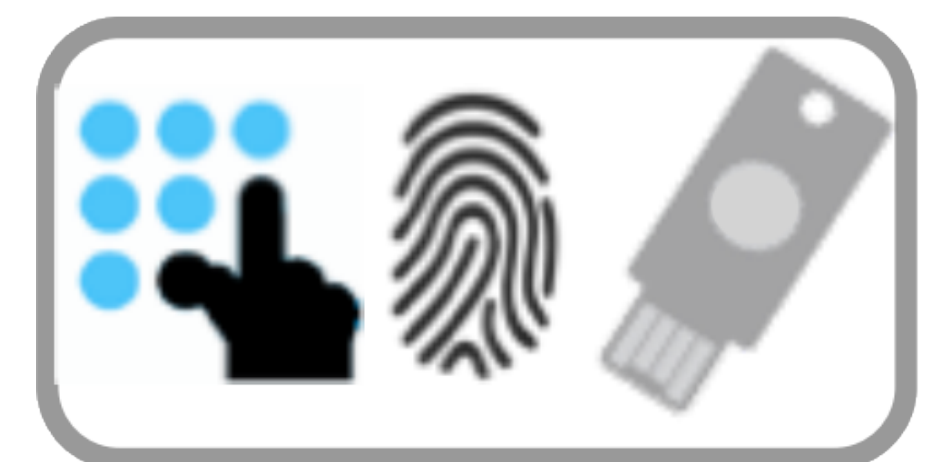
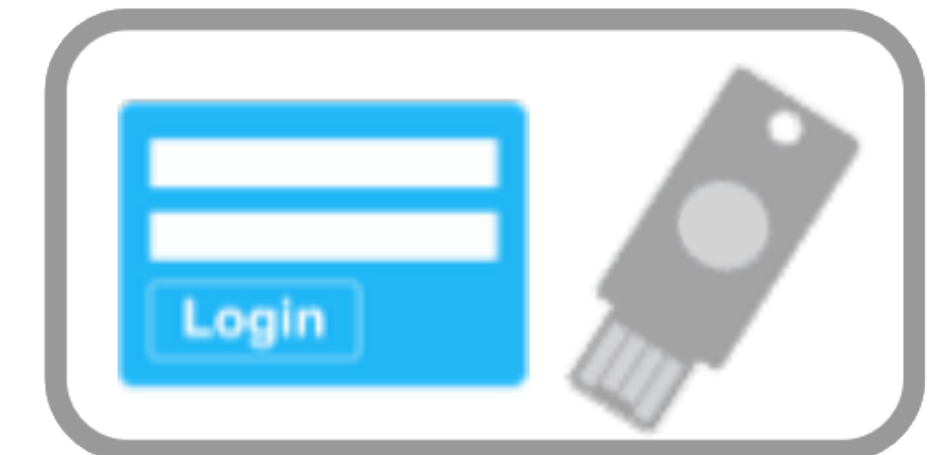
- The challenge is a Nonce
- The response is a signature of the challenge
- + Bob only needs to know Alice's public key
 - Assuming public-key authenticity
- Vulnerable to MitM attacks
 - A man-in-the-middle attacker may proxy the challenge/response and authenticate as Alice



Authentication

Fast Identity Online 2 (FIDO2)

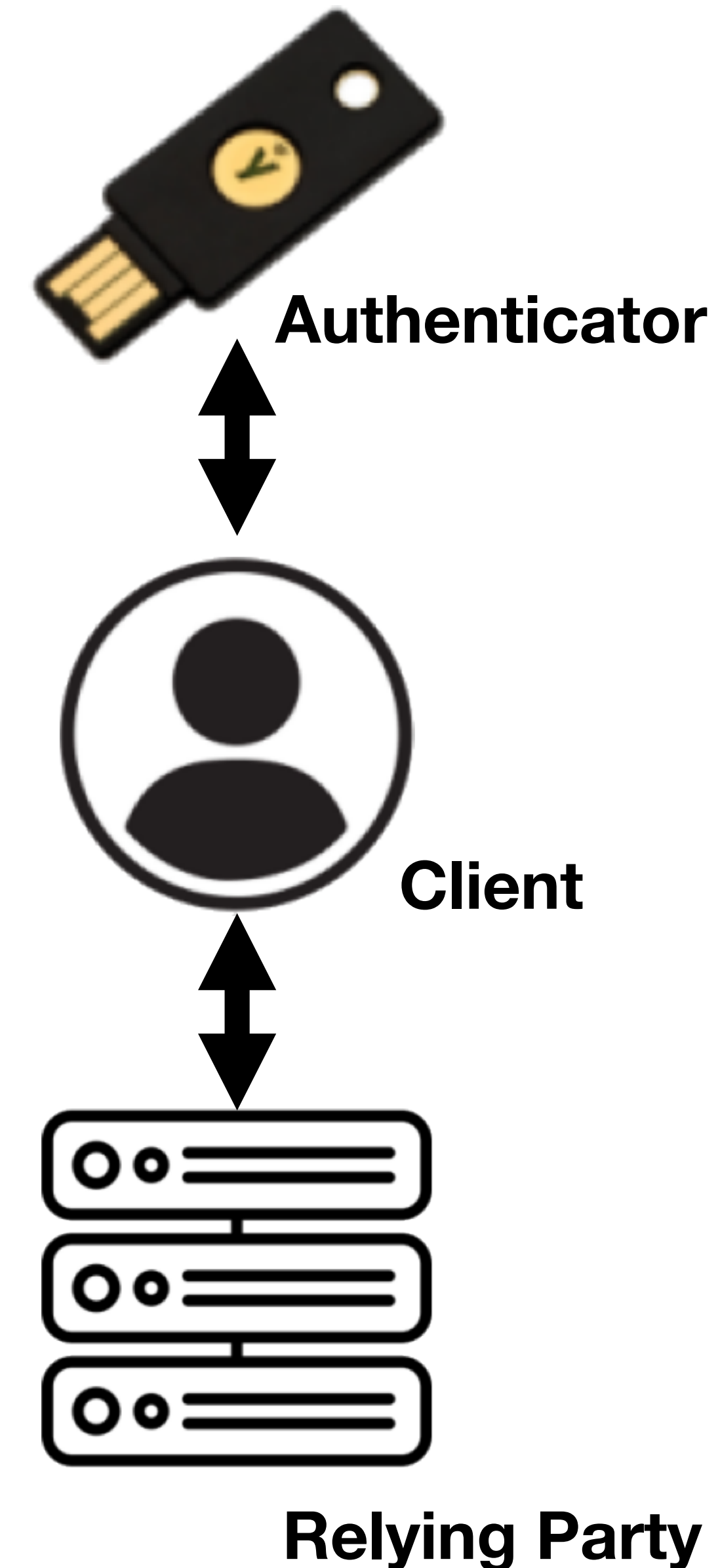
- A W3C standard for authentication based on PKC; evolution of Universal 2nd Factor (U2F)
- **Single Factor:** Passwordless
Replaces weak passwords with strong authentication using hardware authenticator
- **Two Factor:** Password + Authenticator
Second factor in a two factor authentication solution
- **Multi-Factor:** Passwordless + PIN or Biometric
Combination of a hardware authenticator with touch and a PIN for high assurance such as financial transactions



FIDO2

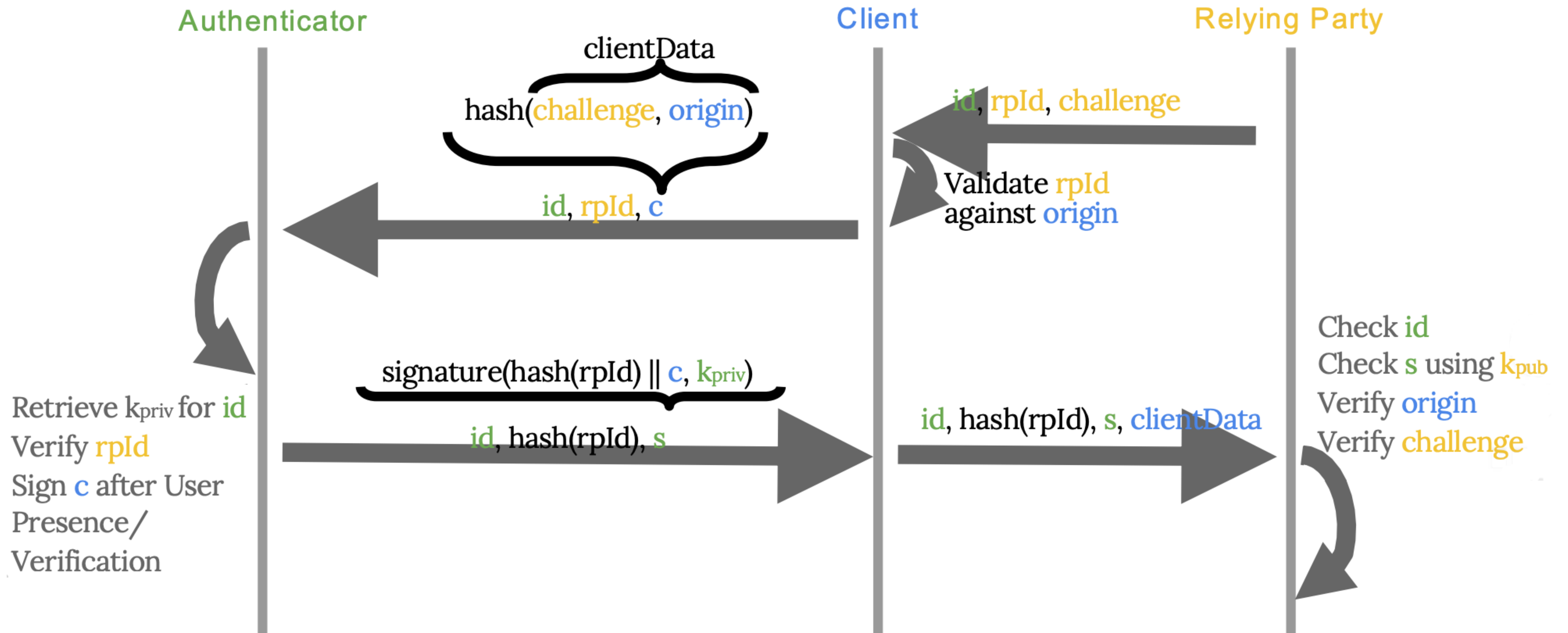
Security

- + Prevents replay
 - Relying party generates unique challenge
- + Prevents phishing (attacker convinces client to give him key)
 - Client validates the origin (e.g. visited web site)
 - Authenticator (on behalf of client) signs challenge plus origin
 - **Binds the signature to that specific origin**
- + Prevents silent tracking and covert login
 - Authenticator checks user presence and consent



FIDO2

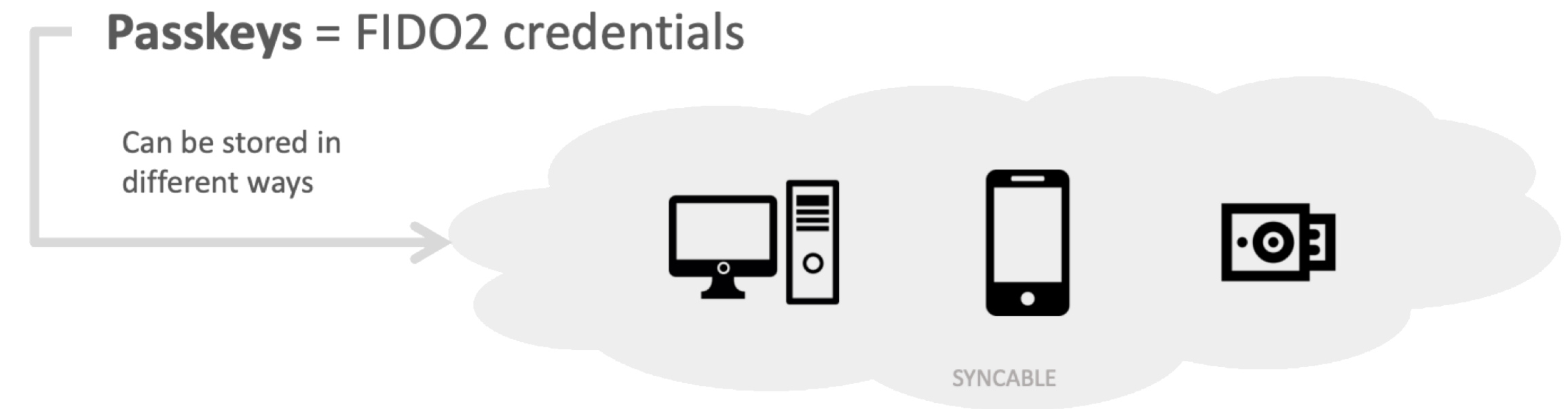
Challenge-response protocol



FIDO2

Usage

- FIDO2 works across devices and platforms, supporting a wide range of authenticators
- The public-private key pair that replaces a password is called a **passkey**
 - Device-Bound Passkeys: the secret key never leaves the authenticator
 - Synced Passkeys: shared across multiple devices and backed up to the cloud (e.g. password manager)



token unlock | unlock the FIDO2 device
token ownership | use the FIDO2 credential

If not synced it's a **device-bound passkeys**



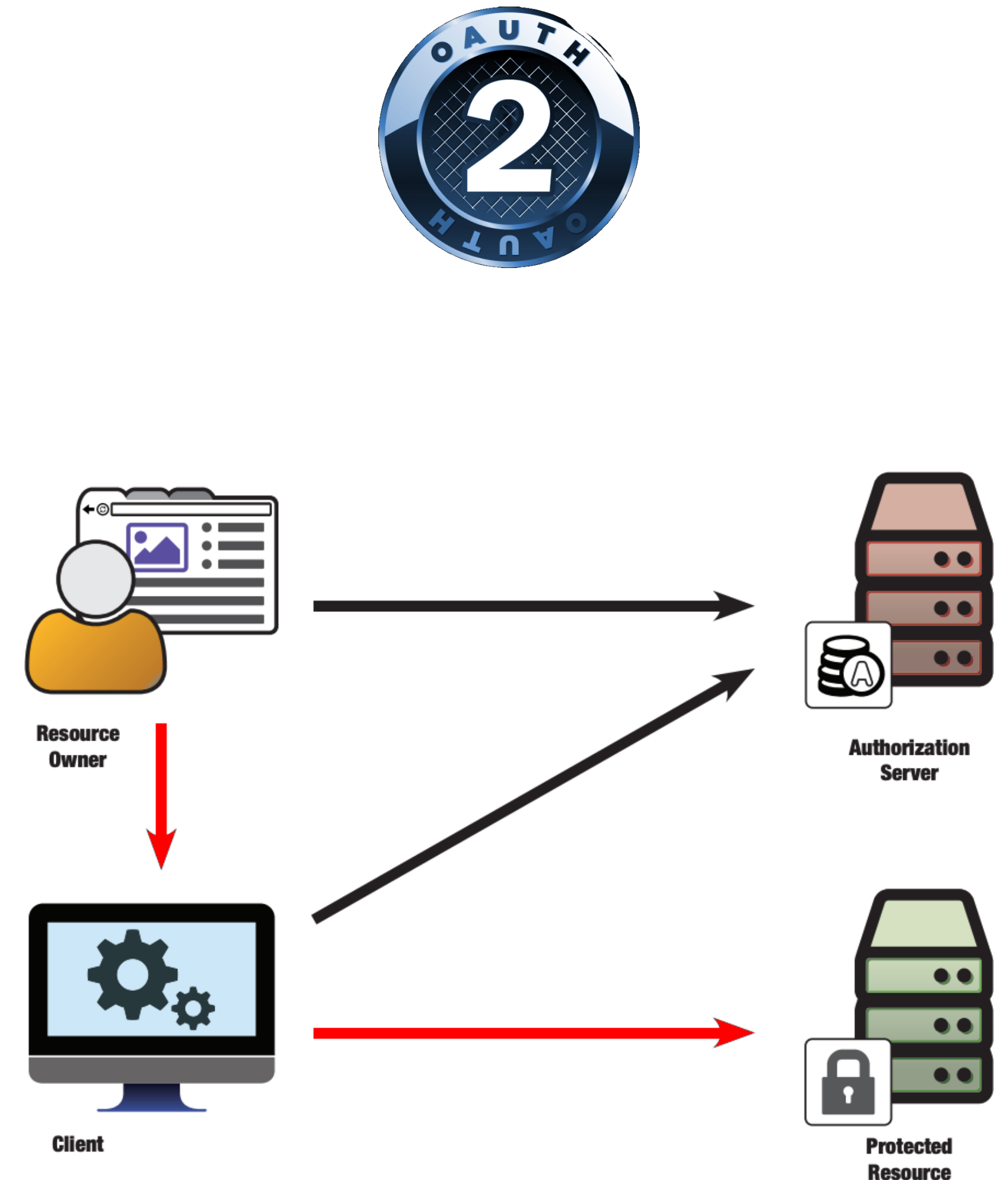
Example: Yubikeys

This block is enclosed in a dashed blue border. It contains text explaining that if passkeys are not synced, they are device-bound. An icon of a Yubikey USB security key is shown next to the text. Below the icon, it gives an example: 'Example: Yubikeys'.

Authorization

OAuth

- **OAuth 2.0** is an IETF standard (RFC 6749)
- From the standard: “enables a **third-party application** to **obtain limited access** to an **HTTP service**, either **on behalf of a resource owner** by orchestrating an approval interaction between the resource owner and the HTTP service, or by allowing the third-party application to obtain access on its own behalf”
- Goal: Give the **client** access to the **protected resource** on behalf of the **resource owner**
 - **Limited access:** issue **client-specific** credentials for the **protected resource**



OAuth Applications

- Not an authentication framework (though extensions exist, e.g. OpenID Connect)
- Used for authorization of services



Click below to change your profile picture!



This will automatically update your Twitter profile picture! We will not post any tweets without your explicit confirmation.



We will not post anything to your Facebook wall without your explicit confirmation.

Authorize Put an Apple On It to use your account?

This application **will be able to:**

- Read Tweets from your timeline.
- See who you follow, and follow new people.
- Update your profile.
- Post Tweets for you.

Authorize app

No, thanks

This application **will not be able to:**

- Access your direct messages.
- See your Twitter password.



Put an Apple On It

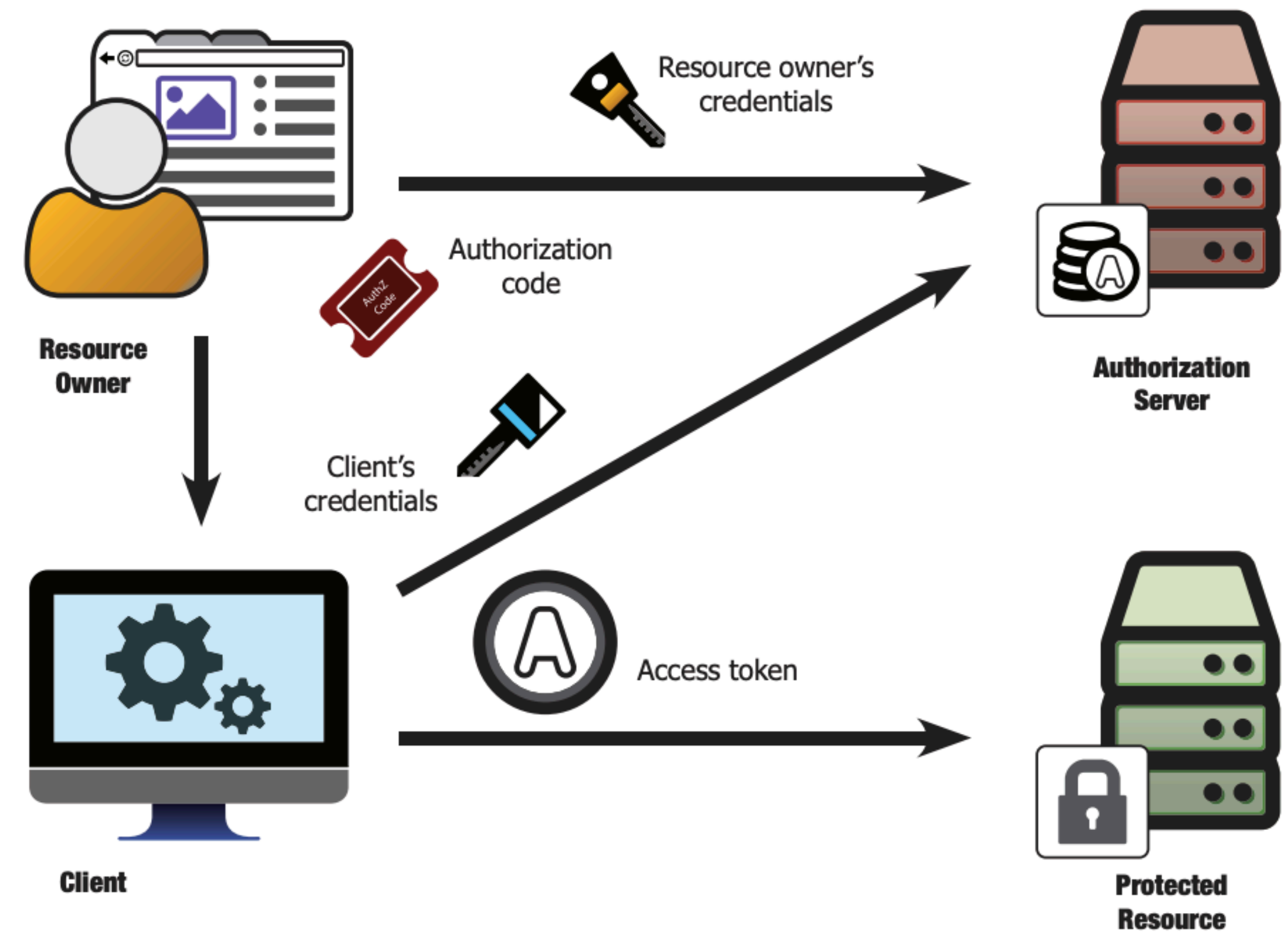
putanappleonit.com

Update your Twitter profile image to show your respect to Steve Jobs

[← Cancel, and return to app](#)

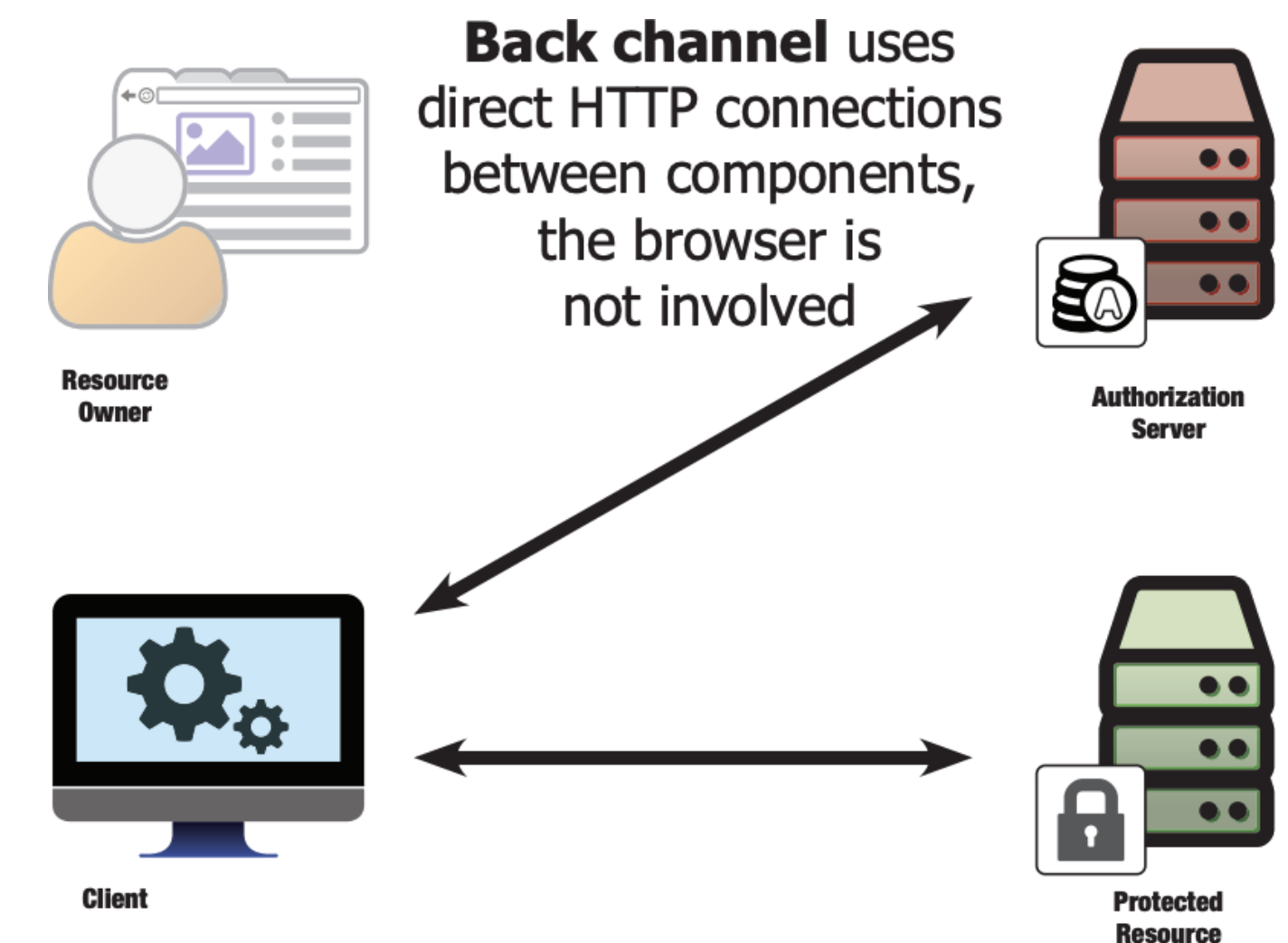
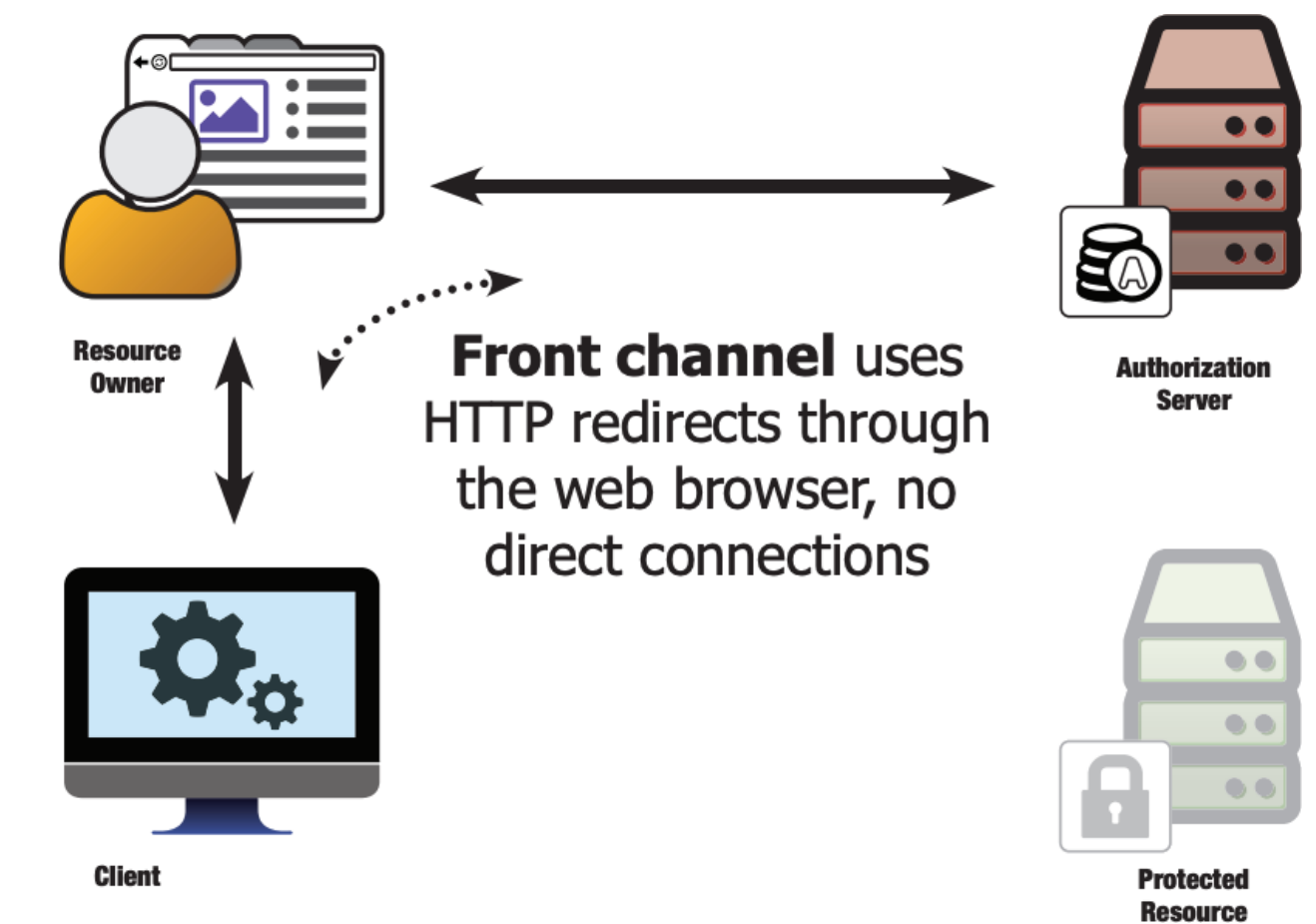
OAuth Architecture

1. **Owner** grants **permission**
 1. Obtains **authorization grant**
 2. Passes it to **client**
2. **Client** accesses **resource**
 1. Exchanges the **authorization grant** for an **access token**
 2. Uses the **access token** to access the **resource**



OAuth Architecture

- Assumes HTTPS for all communication
- Why **authorization grant** $\neq$ **access token**?
 - **Tokens** are **not** encrypted
 - TLS protects data in transit, not from the endpoints!
 - **Owner** communicates via **front channel**
 - **Owner's** browser is a much larger attack vector
 - **Client** communicates via **back channel**
 - **Owner's** browser never sees **token**



OAuth

Scopes

- OAuth scopes limit access to the resource
 - Syntax and semantics not defined in the standard, left to the resource server
- Client can ask for scopes
- Authorization grants and access tokens are bound to scopes

<https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/scopes-for-oauth-apps>

user

- Read/write access to profile info only.

public_repo

- Read/write access to public repos and organizations.

repo

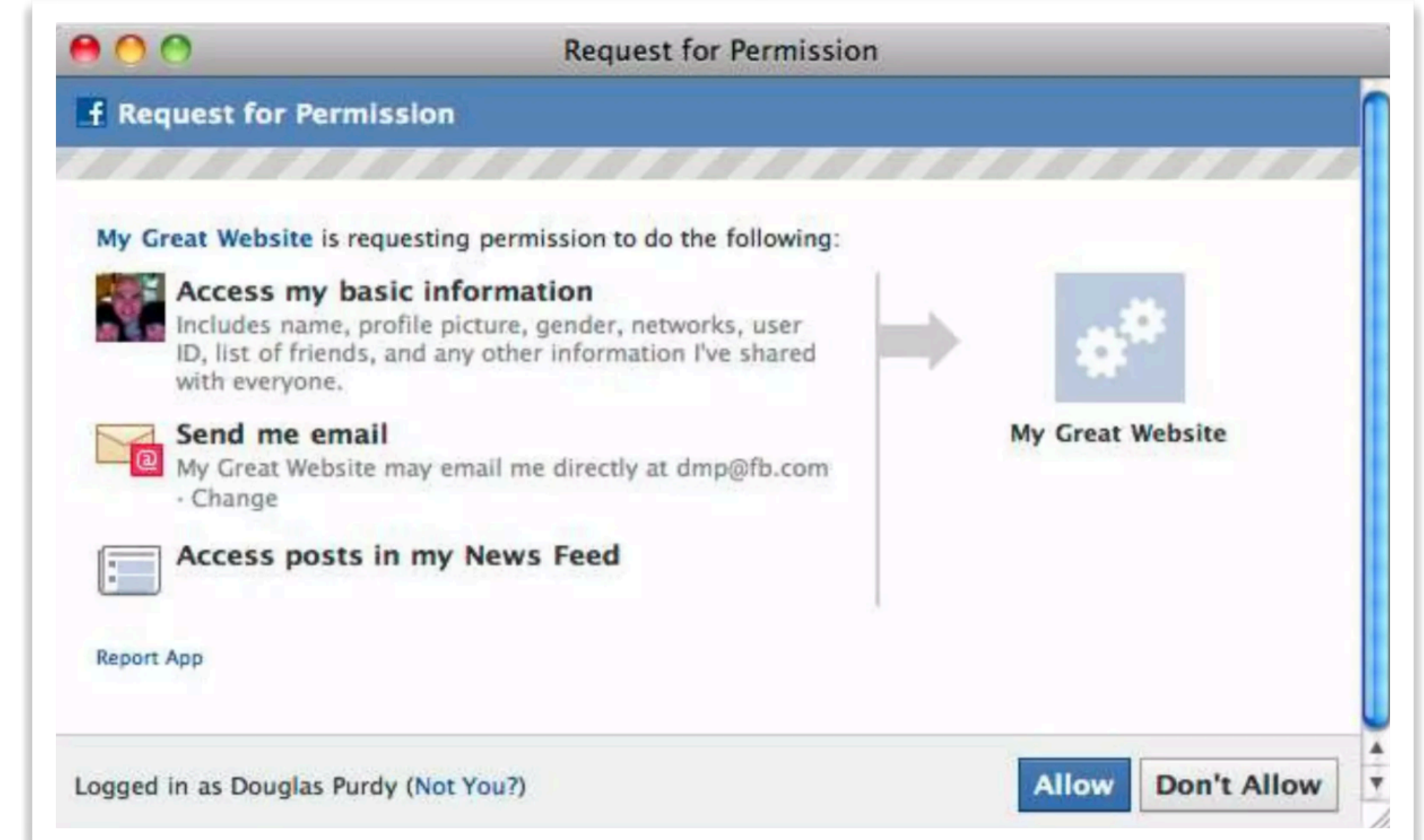
- Read/write access to public and private repos and organizations.

delete_repo

- Delete access to adminable repositories.

gist

- write access to gists.



<https://developers.facebook.com/docs/permissions>

OAuth

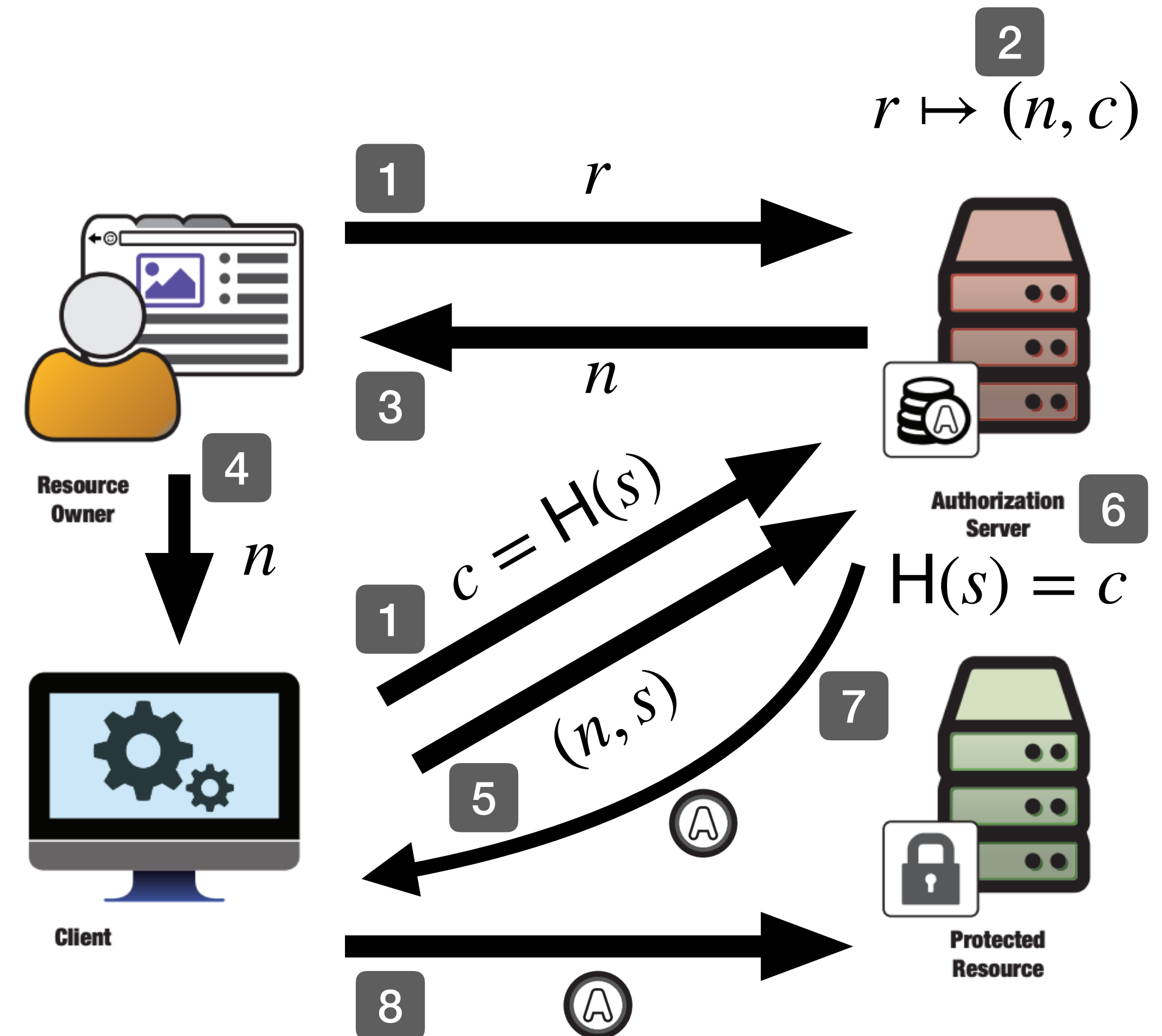
Grants & Tokens

- OAuth grants and tokens are opaque to the client
 - OAuth standard also does not assume any cryptographic methods
 - Grant and token formats are defined by the authentication and resource servers
- However, **JSON Web Tokens (JWT)** is a common format for the token, including:
 - **Header:** the cryptographic algorithm used
 - **Payload:** JSON-encoded content (user ID, expiration, scopes, etc)
 - **Signature:** Binary-encoded result of applying the cryptographic algorithm on the encoded header and payload

OAuth

Authorization Code Grant

- Authorization server generates Nonce n
- Client generates secret s and sends challenge c
- The pair (n, c) is kept as a temporary association to the owner's request r
- Authorization server only releases access token $\textcircled{A}$ if client knows secret s (such that it solves the challenge c)



OAuth

JWT

- Symmetric JWT
 - Authorization server and protected resource are often the same entity
 - Both know shared secret key k (e.g. shared secret from grant phase)
 - $\textcircled{A} = \text{MAC}(k, \text{header} \parallel \text{payload})$
- Asymmetric JWT
 - Separation of token issuer and token verifier
 - Protected resource must know public key of authorization server
 - $\textcircled{A} = \text{Sign}(sk, H(\text{header} \parallel \text{payload}))$

OAuth

Tokens

- Authorization grants are single-use
- Access tokens can be reused (passed around in every access) but should be relatively short-lived (e.g. 1 hour)
- Remember that tokens:
 - Are stateless (and still valid if not expired even after use) $\Rightarrow$ can be **revocated**
 - Can be used even when user is offline $\Rightarrow$ **refresh tokens** support longer sessions
 - Issued alongside the access token, securely stored by client, and used to get new access tokens when access token expires